Reducing Autotuning Cost for GPU Attention Kernels with Workload-Aware Configuration Selection

Status: First draft for mentor review

Target term: Spring 2027, pending renewed confirmation

Faculty supervisor: _____

Scope: Dense prefill attention on NVIDIA GPUs

Research Proposal

Attention is a central operation in modern language models, and its speed depends not only on the algorithm but also on low-level GPU settings such as tile sizes, thread counts, and pipeline stages. Triton makes these settings accessible to programmers, but finding a fast combination commonly requires compiling and benchmarking many candidates. Repeating that exhaustive process for every new workload can make tuning expensive.

This project asks whether simple workload information can identify a small set of promising configurations for previously unseen dense prefill attention workloads. I hypothesize that a lightweight selector can retain at least 90–95% of the performance found by exhaustive tuning while testing no more than 20–25% of a fixed configuration space. The project focuses on dense prefill attention; grouped-query attention, token-by-token decoding, and end-to-end model serving are later extensions.

Existing autotuning systems either perform empirical search or learn cost models from large performance datasets. Prior attention work also makes a simple decision tree an important baseline. However, it remains unclear whether a small ranked candidate set can provide a robust tuning-time/performance tradeoff for unseen dense prefill workloads with limited training data.

My preliminary experiments support this direction. I implemented a forward Triton attention kernel and tested 48 configurations on 16 workloads that vary sequence length, number of attention heads, head dimension, numeric precision, and causal masking. Three complete sweeps on an NVIDIA RTX 3090 produced 696 valid measurements per run, with no correctness failures among configurations that executed successfully. Different workloads preferred different configurations, while exact fastest labels were sometimes unstable across repeated measurements.

I therefore built a selector that returns a short ranked list instead of predicting one winner. On six held-out workloads, a shallow decision-tree selector testing four candidates achieved 96.0% of the repeated-measurement local oracle. Across 100 balanced development/test splits, the same four-candidate budget remained above 90% of the local oracle on every split. In a direct warm-cache experiment, it achieved 96.97% of exhaustive-search performance while reducing total tuning time from 12.600 seconds to 1.987 seconds, an 84.23% reduction. These results are preliminary and apply only to the current GPU, kernel, workloads, configuration space, and cache conditions.

The proposed study will expand and repeat the workload measurements, compare the selector against a fixed default, the best development-set configuration, nearest-workload reuse, random search, and exhaustive tuning, and investigate a stronger learned ranker if the data supports it. Evaluation will measure correctness, performance relative to the local oracle, regret, configurations tested, compilation and benchmarking time, robustness across data splits and random seeds, and the number of repeated executions needed to recover extra tuning cost. If another NVIDIA GPU is available, I will also test hardware transfer.

The expected contribution is a reproducible dataset and an evidence-based evaluation of when a small workload-aware candidate set can replace exhaustive tuning for Triton attention. A negative result would also be useful by identifying workloads or hardware conditions where empirical autotuning remains necessary.